

# INTERNATIONAL UNIVERSITY - VIETNAM NATIONAL UNIVERSITY HCMC



## WEB APPLICATION DEVELOPMENT SCHOOL MANAGEMENT SYSTEM

### GROUP 9 MEMBERS

| MEMBERS            | STUDENTS' ID |
|--------------------|--------------|
| BÙI THỊ THANH NGÂN | ITCSIU24061  |
| PHẠM QUANG THỊNH   | ITDSIU24047  |
| TRẦN MINH PHÚC     | ITCSIU24070  |

Contents

---

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                               | <b>5</b>  |
| 1.1      | Project Overview . . . . .                             | 5         |
| 1.2      | Problem Statement . . . . .                            | 5         |
| 1.3      | Target Users & Key Features . . . . .                  | 5         |
| 1.4      | Technology Stack Summary . . . . .                     | 6         |
| 1.5      | Test Credentials . . . . .                             | 6         |
| <b>2</b> | <b>Requirements Analysis</b>                           | <b>6</b>  |
| 2.1      | Problem Statement . . . . .                            | 6         |
| 2.2      | User Stories . . . . .                                 | 7         |
| 2.3      | Functional Requirements . . . . .                      | 8         |
| 2.4      | Non-Functional Requirements . . . . .                  | 9         |
| <b>3</b> | <b>System Architecture</b>                             | <b>10</b> |
| 3.1      | Architecture Overview . . . . .                        | 10        |
| 3.2      | Technology Stack Justification . . . . .               | 11        |
| 3.2.1    | Backend: Java Servlets + JSP vs. Spring Boot . . . . . | 11        |
| 3.2.2    | Database: MySQL 8 vs. PostgreSQL . . . . .             | 11        |
| 3.2.3    | Frontend: Tailwind CSS vs. Bootstrap . . . . .         | 12        |
| 3.2.4    | Hosting: Local / Tomcat deployment . . . . .           | 12        |
| 3.3      | Design Patterns Used . . . . .                         | 12        |
| 3.4      | API Design . . . . .                                   | 13        |
| <b>4</b> | <b>Database Design</b>                                 | <b>13</b> |
| 4.1      | Entity-Relationship Overview . . . . .                 | 13        |
| 4.2      | Table Schemas . . . . .                                | 13        |
| 4.2.1    | Roles . . . . .                                        | 13        |
| 4.2.2    | Users . . . . .                                        | 14        |
| 4.2.3    | Students . . . . .                                     | 14        |
| 4.2.4    | Teachers . . . . .                                     | 14        |
| 4.2.5    | Classes . . . . .                                      | 15        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 4.2.6    | Rooms . . . . .                                       | 15        |
| 4.2.7    | Subjects . . . . .                                    | 15        |
| 4.2.8    | TeachingAssignments (Junction Table) . . . . .        | 15        |
| 4.2.9    | ClassSchedules . . . . .                              | 16        |
| 4.2.10   | StudentFees . . . . .                                 | 16        |
| 4.2.11   | TeacherPayroll . . . . .                              | 16        |
| 4.2.12   | RememberTokens . . . . .                              | 17        |
| 4.2.13   | AuditLogs . . . . .                                   | 17        |
| 4.3      | Relationships . . . . .                               | 17        |
| 4.4      | Normalization Analysis . . . . .                      | 18        |
| 4.5      | Indexing Strategy . . . . .                           | 18        |
| <b>5</b> | <b>Implementation Details</b>                         | <b>19</b> |
| 5.1      | Authentication System . . . . .                       | 19        |
| 5.1.1    | Login Flow & Session Fixation Protection . . . . .    | 19        |
| 5.1.2    | BCrypt Password Hashing . . . . .                     | 19        |
| 5.1.3    | Remember Me Implementation . . . . .                  | 20        |
| 5.2      | Role-Based Access Control (RBAC) . . . . .            | 20        |
| 5.3      | Payroll Business Logic . . . . .                      | 21        |
| 5.3.1    | Automatic Net Amount Computation . . . . .            | 21        |
| 5.3.2    | Duplicate Payroll Prevention . . . . .                | 22        |
| 5.3.3    | Automated Status Transition via MySQL Event . . . . . | 22        |
| 5.4      | Student Fee Payment with Security Guard . . . . .     | 22        |
| 5.5      | OVERDUE Status: Virtual Computation . . . . .         | 23        |
| 5.6      | Post/Redirect/Get Pattern . . . . .                   | 24        |
| 5.7      | Server-Side Form Validation . . . . .                 | 24        |
| 5.8      | AI Assistance Report . . . . .                        | 25        |
| 5.8.1    | Tools Used . . . . .                                  | 25        |
| 5.8.2    | Authentication System . . . . .                       | 25        |
| 5.8.3    | Remember Me Feature . . . . .                         | 26        |
| 5.8.4    | MySQL Scheduled Event . . . . .                       | 26        |
| 5.8.5    | Challenges with AI Code . . . . .                     | 27        |
| 5.8.6    | Overall Impact . . . . .                              | 27        |

|           |                                                 |           |
|-----------|-------------------------------------------------|-----------|
| <b>6</b>  | <b>Security Implementation</b>                  | <b>27</b> |
| 6.1       | Password Hashing . . . . .                      | 27        |
| 6.2       | SQL Injection Prevention . . . . .              | 27        |
| 6.3       | XSS Prevention . . . . .                        | 28        |
| 6.4       | CSRF Protection . . . . .                       | 28        |
| 6.5       | Authentication Security . . . . .               | 28        |
| 6.6       | Authorisation . . . . .                         | 28        |
| <b>7</b>  | <b>Testing and Quality Assurance</b>            | <b>28</b> |
| 7.1       | Testing Strategy . . . . .                      | 28        |
| 7.2       | Bug Tracking . . . . .                          | 30        |
| 7.3       | Known Limitations . . . . .                     | 30        |
| <b>8</b>  | <b>Deployment</b>                               | <b>31</b> |
| 8.1       | Deployment Process . . . . .                    | 31        |
| 8.1.1     | Prerequisites . . . . .                         | 31        |
| 8.1.2     | Step-by-Step Setup . . . . .                    | 31        |
| 8.2       | CI/CD Pipeline . . . . .                        | 33        |
| 8.3       | Monitoring and Maintenance . . . . .            | 33        |
| <b>9</b>  | <b>User Guide</b>                               | <b>33</b> |
| 9.1       | Getting Started . . . . .                       | 33        |
| 9.2       | Student: Fee Payment Portal . . . . .           | 34        |
| 9.3       | Admin: Payroll Management . . . . .             | 34        |
| 9.4       | Teacher / Admin: Financial Statistics . . . . . | 34        |
| <b>10</b> | <b>Team Collaboration</b>                       | <b>34</b> |
| 10.1      | Team Organisation . . . . .                     | 35        |
| 10.2      | Individual Contributions . . . . .              | 35        |
| 10.3      | Challenges in Teamwork . . . . .                | 35        |
| <b>11</b> | <b>Conclusion and Reflection</b>                | <b>35</b> |
| 11.1      | Project Summary . . . . .                       | 36        |
| 11.2      | Lessons Learned . . . . .                       | 36        |
| 11.3      | Future Enhancements . . . . .                   | 36        |

|                                         |           |
|-----------------------------------------|-----------|
| <b>Appendix A: Endpoint Reference</b>   | <b>38</b> |
| <b>Appendix B: Repository Structure</b> | <b>39</b> |

## 1. Executive Summary

---

### 1.1. Project Overview

The **School Management System** is a full-stack web application developed as the final project for the Web Application Development course at International University – VNU HCMC. The system addresses the real-world administrative challenge of managing student fee payments, teacher payroll, and school financial reporting through a single, role-aware platform.

### 1.2. Problem Statement

Traditional school administration relies on spreadsheets and disconnected tools to track fee payments and teacher salaries. This creates data inconsistencies, delayed payroll disbursements, and a lack of real-time financial visibility for administrators. Our system centralises these workflows into a secure, browser-based application accessible to all stakeholders.

### 1.3. Target Users & Key Features

- **Admin** – generates and manages teacher payroll, views financial dashboards, administers all users.
- **Teacher** – views financial statistics and salary information for the school.
- **Student** – views personal fee records, tracks outstanding balances, and submits payments.

Core features delivered:

- Role-based access control (RBAC) with three distinct user roles
- Secure authentication: BCrypt hashing, session fixation protection, CSRF token validation, and persistent “Remember Me” via database-backed tokens
- Student fee portal with PAID / UNPAID / OVERDUE status lifecycle
- Teacher payroll management: generate, edit, update, and delete payroll records; net pay computed automatically by the database

- Automated payroll status transitions via a MySQL scheduled event
- Financial statistics dashboard for admins and teachers
- Server-side input validation on all forms
- Responsive UI built with Tailwind CSS

#### 1.4. Technology Stack Summary

| Layer            | Technology                                 |
|------------------|--------------------------------------------|
| Language         | Java 17+                                   |
| Web Framework    | Jakarta Servlet API 6.0, JSP 3.1, JSTL 3.0 |
| Build Tool       | Apache Maven                               |
| Server           | Apache Tomcat 10+                          |
| Database         | MySQL 8+                                   |
| Password Hashing | jBCrypt 0.4                                |
| Frontend Styling | Tailwind CSS 3.4                           |
| JDBC Driver      | MySQL Connector/J 8.2.0                    |

#### 1.5. Test Credentials

| Role    | Username     | Password        |
|---------|--------------|-----------------|
| Admin   | admin        | 123admin*       |
| Teacher | teacher_phuc | 123teacherphuc* |
| Student | student_huy  | 123studenthuy*  |

## 2. Requirements Analysis

### 2.1. Problem Statement

Schools routinely struggle with fragmented systems for tracking student tuition fees and managing teacher payroll. Fee records stored in spreadsheets lead to missed due dates; payroll handled manually creates delays and errors. There is also a lack of a unified, role-aware interface – teachers cannot quickly see school-level financial data,

and students cannot self-serve their fee status without contacting the administrative office.

Our application solves these problems by providing:

- A single login portal that routes each user to their appropriate dashboard
- Real-time fee status for students (PAID / UNPAID / OVERDUE)
- Automated monthly payroll generation for administrators
- An automated hourly MySQL event that escalates overdue-pending payrolls to ON\_HOLD
- A financial statistics overview for teachers and administrators

## 2.2. User Stories

| # | User Story                                                                                                               | Priority  |
|---|--------------------------------------------------------------------------------------------------------------------------|-----------|
| 1 | As a <b>student</b> , I want to log in securely so that only I can view my fee records.                                  | Must Have |
| 2 | As a <b>student</b> , I want to see all my fee records with their due dates and amounts so that I know what I owe.       | Must Have |
| 3 | As a <b>student</b> , I want to mark a fee as paid so that my balance is updated immediately.                            | Must Have |
| 4 | As a <b>student</b> , I want to see which fees are overdue so that I can prioritise payments.                            | Must Have |
| 5 | As a <b>student</b> , I want to register an account using my student code so that I can access the system.               | Must Have |
| 6 | As a <b>teacher</b> , I want to log in and view financial statistics so that I understand the school's financial health. | Must Have |
| 7 | As a <b>teacher</b> , I want to register an account using my teacher code so that I can access the system.               | Must Have |
| 8 | As an <b>admin</b> , I want to generate monthly payroll for all teachers so that salaries are prepared on time.          | Must Have |
| 9 | As an <b>admin</b> , I want to edit payroll records (base salary, allowances, deductions) so that I can correct errors.  | Must Have |



| #  | User Story                                                                                                                                          | Priority    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 10 | As an <b>admin</b> , I want to update a payroll's status (PENDING / DISBURSED / ON_HOLD) so that disbursement state is tracked.                     | Must Have   |
| 11 | As an <b>admin</b> , I want to delete a payroll record so that erroneous entries can be removed.                                                    | Must Have   |
| 12 | As an <b>admin</b> , I want the system to prevent duplicate payroll generation for the same period so that data integrity is maintained.            | Must Have   |
| 13 | As an <b>admin</b> , I want overdue-pending payrolls to be automatically escalated to ON_HOLD so that I do not miss missed payments.                | Should Have |
| 14 | As an <b>admin</b> , I want a dashboard showing total payroll, disbursed count, pending count, and on-hold count so that I have at-a-glance status. | Should Have |
| 15 | As <b>any user</b> , I want clear error messages when I submit invalid form data so that I can correct mistakes.                                    | Must Have   |
| 16 | As <b>any user</b> , I want to use "Remember Me" so that I stay logged in across browser sessions.                                                  | Should Have |
| 17 | As <b>any user</b> , I want to log out securely (with CSRF protection) so that my account is not hijacked.                                          | Must Have   |
| 18 | As an <b>admin</b> , I want to filter payroll records by pay period so that I can view historical payroll data.                                     | Should Have |
| 19 | As a <b>student</b> , I want a running total of my outstanding debt so that I know my total balance at a glance.                                    | Should Have |
| 20 | As <b>any user</b> , I want the application to work correctly on mobile and desktop so that I can use it anywhere.                                  | Should Have |

### 2.3. Functional Requirements

1. **Authentication & Session Management** – Users must log in with username and password. Passwords must be hashed using BCrypt. Sessions must be invalidated on logout. CSRF tokens must protect state-changing operations.

2. **Role-Based Access Control** – Three roles (ADMIN, TEACHER, STUDENT) must have distinct landing pages and access permissions enforced by servlet filter chains.
3. **User Registration** – New users register by providing their existing teacher/student code, a username, a password, and an email. Role selection drives which entity lookup is performed.
4. **Student Fee Portal** – Students view all fee records for their account. Each record displays amount, due date, payment date, and status. Students can submit payment for any UNPAID fee.
5. **Teacher Payroll Management** – Admins generate payroll for the current month. Each teacher receives one payroll record seeded from their base salary. Admins can edit, update, and delete individual records.
6. **Automated Status Transitions** – A MySQL scheduled event runs hourly and transitions PENDING payroll records with a past payment date to ON\_HOLD.
7. **Financial Statistics Dashboard** – Admins and teachers view a summary page showing fee collection and payroll metrics.
8. **Input Validation** – All forms perform server-side validation with field-level error messages returned to the user.
9. **Remember Me** – Users who check “Remember Me” receive an HttpOnly cookie containing a token that restores their session on the next visit for up to 30 days.

## 2.4. Non-Functional Requirements

- **Performance:** Pages should load within 2 seconds on a local deployment. Database queries use parameterized statements.
- **Security:** Passwords stored with BCrypt (never plain-text). SQL injection prevented via PreparedStatement. Session fixation protection on login. CSRF token on logout. Generic error messages prevent username enumeration.
- **Usability:** Responsive layout works on mobile, tablet, and desktop. Form errors are shown inline next to the offending field. Flash messages confirm or reject actions after redirects.

- **Reliability:** Custom 404 page for unknown routes. Database errors are wrapped in `ServletException` and handled gracefully.
- **Maintainability:** Clear separation of concerns: Controller → DAO → Model → View (MVC). No business logic in JSP pages.

### 3. System Architecture

---

#### 3.1. Architecture Overview

The application is a **server-rendered MVC web application** built on the Jakarta EE Servlet stack. There is no client-side JavaScript framework and no JSON REST API – the browser communicates exclusively through HTML form submissions and GET requests. The server renders complete HTML pages via JSP and returns them to the browser.

Browser

HTML form POST / GET



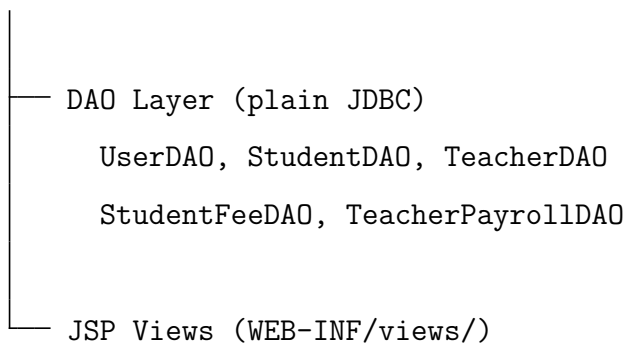
Tomcat 10 (Jakarta Servlet Container)

Filter Chain

|               |                         |                            |
|---------------|-------------------------|----------------------------|
| AuthFilter    | (/*)                    | -- auth gate + Remember Me |
| AdminFilter   | (/payroll)              | -- admin-only guard        |
| TeacherFilter | (/financial-statistics) | -- admin+teacher guard     |
| StudentFilter | (/payment)              | -- student-only guard      |

Servlet Controllers

|                              |                       |
|------------------------------|-----------------------|
| LoginController              | /login                |
| LogoutController             | /logout               |
| RegisterController           | /register             |
| PaymentController            | /payment              |
| PayrollController            | /payroll              |
| FinancialStatisticController | /financial-statistics |



MySQL 8 (school\_management database)

Data flows in one direction per request: the browser sends a request, a filter validates authentication and authorisation, the appropriate servlet controller handles the request, delegates persistence work to DAOs, populates request/session attributes, and either forwards to a JSP view or issues a redirect.

## 3.2. Technology Stack Justification

### 3.2.1. Backend: Java Servlets + JSP vs. Spring Boot

We chose raw Jakarta Servlets and JSP over Spring Boot deliberately, as this project follows *Labs 01–08* of the course which build servlet-level understanding progressively. Using Spring Boot would have abstracted away the component lifecycle, filter chains, and request dispatching that were explicitly covered in lectures. By staying with plain servlets, every line of authentication, RBAC, and session handling is code we wrote and can explain.

The trade-off is more boilerplate: we handle connection management, URL mapping, and validation manually rather than relying on Spring’s auto-wiring. This was an acceptable cost given the educational objective.

### 3.2.2. Database: MySQL 8 vs. PostgreSQL

MySQL 8 was chosen because it is pre-installed in the university lab environment and is the database covered in the course labs. It supports generated columns (used for `net_amount` in the `TeacherPayroll` table), scheduled events (used for automatic status transitions), and has mature JDBC driver support.

PostgreSQL would be the preferred choice in a production context for its stronger

standards compliance and better JSON support, but MySQL met all requirements for this project and reduced setup friction for all team members.

### 3.2.3. Frontend: Tailwind CSS vs. Bootstrap

Tailwind CSS was chosen over Bootstrap because it produces smaller output bundles (only utility classes actually used are included), avoids the default Bootstrap appearance, and gives full control over design without needing to override component styles. The Tailwind CLI is integrated into the project's `frontend/` package.

All interactive functionality (form toggling on the registration page, flash message auto-dismissal) is implemented in vanilla JavaScript. No JS framework was used – this was consistent with the server-rendered architecture.

### 3.2.4. Hosting: Local / Tomcat deployment

The application is packaged as a `.war` file deployed to Apache Tomcat. For production deployment, it can be containerised with Docker (a `docker-compose.yml` is noted as a future improvement in the repository).

## 3.3. Design Patterns Used

- **Model-View-Controller (MVC):** Servlet controllers handle request routing and delegate to DAOs (model layer), then forward to JSP views. Views contain no business logic.
- **Data Access Object (DAO):** Each DAO class encapsulates all SQL for one aggregate root (`UserDAO`, `StudentDAO`, etc.), providing a clean interface between the controller and the database.
- **Filter Chain:** Cross-cutting concerns (authentication, authorisation) are handled in a chain of servlet filters, keeping controllers free of security boilerplate.
- **Post/Redirect/Get (PRG):** All POST handlers redirect after a successful operation. Flash messages stored in the session carry success/error feedback across the redirect, preventing form re-submission on browser refresh.
- **Action Dispatch:** Controllers managing multiple sub-operations read a `?action=` query parameter and switch on it, reducing the need for multiple servlet mappings.

### 3.4. API Design

The application is not a REST API – it uses traditional web MVC. Key endpoints are listed below for reference:

| URL Pattern           | Methods   | Role           |
|-----------------------|-----------|----------------|
| /login                | GET, POST | Public         |
| /logout               | GET, POST | Authenticated  |
| /register             | GET, POST | Public         |
| /payment              | GET, POST | STUDENT        |
| /payroll              | GET, POST | ADMIN          |
| /financial-statistics | GET       | ADMIN, TEACHER |

## 4. Database Design

### 4.1. Entity-Relationship Overview

The `school_management` database contains 13 tables covering academic structure, user management, financial transactions, and system auditing. The schema is available in `database/schema.sql`; the ER diagram is in `database/ER-diagram.png`.

### 4.2. Table Schemas

#### 4.2.1. Roles

| Column                   | Type        | Constraints                 | Notes             |
|--------------------------|-------------|-----------------------------|-------------------|
| <code>role_id</code>     | INT         | PRIMARY KEY, AUTO_INCREMENT |                   |
| <code>role_name</code>   | VARCHAR(50) | UNIQUE NOT NULL             | ADMIN, TEACHER, S |
| <code>description</code> | TEXT        |                             |                   |

## 4.2.2. Users

| Column        | Type         | Constraints                 | Notes        |
|---------------|--------------|-----------------------------|--------------|
| user_id       | INT          | PRIMARY KEY, AUTO_INCREMENT |              |
| username      | VARCHAR(100) | UNIQUE NOT NULL             |              |
| password_hash | VARCHAR(255) | NOT NULL                    | BCrypt hash  |
| email         | VARCHAR(255) | UNIQUE NOT NULL             |              |
| role_id       | INT          | FK → Roles                  |              |
| is_active     | TINYINT(1)   | DEFAULT 1                   | soft-disable |
| created_at    | TIMESTAMP    | DEFAULT NOW()               | audit        |
| last_login    | TIMESTAMP    |                             |              |

## 4.2.3. Students

| Column       | Type         | Constraints                 | Notes                   |
|--------------|--------------|-----------------------------|-------------------------|
| student_id   | INT          | PRIMARY KEY, AUTO_INCREMENT |                         |
| student_code | VARCHAR(20)  | UNIQUE NOT NULL             | e.g. AI001              |
| student_name | VARCHAR(100) | NOT NULL                    |                         |
| major        | VARCHAR(100) |                             |                         |
| email        | VARCHAR(255) | UNIQUE                      |                         |
| class_id     | INT          | FK → Classes                |                         |
| user_id      | INT          | FK → Users, UNIQUE          | nullable until register |

## 4.2.4. Teachers

| Column       | Type          | Constraints                 | Notes                       |
|--------------|---------------|-----------------------------|-----------------------------|
| teacher_id   | INT           | PRIMARY KEY, AUTO_INCREMENT |                             |
| teacher_code | VARCHAR(20)   | UNIQUE NOT NULL             | e.g. TSE001                 |
| teacher_name | VARCHAR(100)  | NOT NULL                    |                             |
| email        | VARCHAR(255)  | UNIQUE                      |                             |
| phone        | VARCHAR(20)   |                             |                             |
| salary       | DECIMAL(12,2) | NOT NULL                    | base for payroll generation |
| user_id      | INT           | FK → Users, UNIQUE          | nullable until register     |

#### 4.2.5. Classes

| Column     | Type        | Constraints                 |
|------------|-------------|-----------------------------|
| class_id   | INT         | PRIMARY KEY, AUTO_INCREMENT |
| class_name | VARCHAR(50) | NOT NULL                    |
| section    | VARCHAR(20) |                             |
| room_id    | INT         | FK → Rooms                  |

e.g. SE2024A

#### 4.2.6. Rooms

| Column    | Type        | Constraints                 |
|-----------|-------------|-----------------------------|
| room_id   | INT         | PRIMARY KEY, AUTO_INCREMENT |
| room_name | VARCHAR(50) | NOT NULL                    |
| room_type | ENUM        | CLASSROOM, LAB              |

#### 4.2.7. Subjects

| Column       | Type         | Constraints                 |
|--------------|--------------|-----------------------------|
| subject_id   | INT          | PRIMARY KEY, AUTO_INCREMENT |
| subject_code | VARCHAR(20)  | UNIQUE NOT NULL             |
| subject_name | VARCHAR(100) | NOT NULL                    |
| credits      | INT          | CHECK (> 0)                 |

#### 4.2.8. TeachingAssignments (Junction Table)

Maps teachers to classes and subjects (many-to-many between teachers and subjects via classes).

| Column        | Type | Constraints                 |
|---------------|------|-----------------------------|
| assignment_id | INT  | PRIMARY KEY, AUTO_INCREMENT |
| class_id      | INT  | FK → Classes                |
| subject_id    | INT  | FK → Subjects               |
| teacher_id    | INT  | FK → Teachers               |



## 4.2.9. ClassSchedules

| Column      | Type | Constraints                 |
|-------------|------|-----------------------------|
| schedule_id | INT  | PRIMARY KEY, AUTO_INCREMENT |
| class_id    | INT  | FK → Classes                |
| subject_id  | INT  | FK → Subjects               |
| room_id     | INT  | FK → Rooms                  |
| day_of_week | ENUM | MONDAY...SATURDAY           |
| start_time  | TIME | NOT NULL                    |
| end_time    | TIME | NOT NULL                    |

## 4.2.10. StudentFees

| Column       | Type          | Constraints                 | Notes              |
|--------------|---------------|-----------------------------|--------------------|
| fee_id       | INT           | PRIMARY KEY, AUTO_INCREMENT |                    |
| student_id   | INT           | FK → Students, NOT NULL     |                    |
| amount       | DECIMAL(12,2) | NOT NULL                    |                    |
| due_date     | DATE          | NOT NULL                    |                    |
| payment_date | DATE          |                             | set when paid      |
| status       | ENUM          | PAID, UNPAID                | OVERDUE is virtual |

## 4.2.11. TeacherPayroll

| Column       | Type          | Constraints                 | Notes                  |
|--------------|---------------|-----------------------------|------------------------|
| payroll_id   | INT           | PRIMARY KEY, AUTO_INCREMENT |                        |
| teacher_id   | INT           | FK → Teachers, NOT NULL     |                        |
| pay_period   | VARCHAR(7)    | NOT NULL                    | YYYY-MM                |
| base_salary  | DECIMAL(12,2) | NOT NULL                    |                        |
| allowances   | DECIMAL(12,2) | DEFAULT 0.00                |                        |
| deductions   | DECIMAL(12,2) | DEFAULT 0.00                |                        |
| net_amount   | DECIMAL(12,2) | GENERATED ALWAYS AS ...     | base + allow – ded     |
| payment_date | DATE          |                             | scheduled disbursement |
| status       | ENUM          | PENDING, DISBURSED, ON_HOLD |                        |

#### 4.2.12. RememberTokens

| Column     | Type         | Constraints                 |
|------------|--------------|-----------------------------|
| token_id   | INT          | PRIMARY KEY, AUTO_INCREMENT |
| user_id    | INT          | FK → Users                  |
| token_hash | VARCHAR(255) | UNIQUE NOT NULL             |
| expires_at | TIMESTAMP    | NOT NULL                    |
| created_at | TIMESTAMP    | DEFAULT NOW()               |

#### 4.2.13. AuditLogs

| Column      | Type        | Constraints                 |
|-------------|-------------|-----------------------------|
| log_id      | INT         | PRIMARY KEY, AUTO_INCREMENT |
| user_id     | INT         | FK → Users, nullable        |
| action_type | VARCHAR(50) | NOT NULL                    |
| description | TEXT        |                             |
| ip_address  | VARCHAR(50) |                             |
| created_at  | TIMESTAMP   | DEFAULT NOW()               |

### 4.3. Relationships

- **Users – Roles:** Many-to-one. Each user belongs to exactly one role. Enforced by `role_id` FK in Users.
- **Users – Students / Teachers:** One-to-one (optional). A student or teacher record may exist without a user account (pre-seeded by admin) and gains a `user_id` link only upon registration.
- **Students – Classes:** Many-to-one. Each student belongs to one class; a class contains many students.
- **Classes – Rooms:** Many-to-one. Each class is assigned a home room.
- **Teachers – Subjects (via TeachingAssignments):** Many-to-many. A teacher may teach multiple subjects in multiple classes; a subject/class combination may be taught by a teacher. `TeachingAssignments` is the junction table.

- **Classes – Subjects (via ClassSchedules):** A class may have multiple schedule slots; each slot links a class, subject, room, day, and time window.
- **Students – StudentFees:** One-to-many. Each student has many fee records over time.
- **Teachers – TeacherPayroll:** One-to-many. Each teacher has one payroll record per pay period.
- **Users – RememberTokens:** One-to-many. A user may have multiple active tokens (from different devices).

#### 4.4. Normalization Analysis

The schema is in **Third Normal Form (3NF)**:

- **1NF:** All columns contain atomic values. No repeating groups.
- **2NF:** All non-key attributes are fully functionally dependent on the primary key. Junction tables (**TeachingAssignments**, **ClassSchedules**) have composite-key semantics handled via a surrogate primary key.
- **3NF:** No transitive dependencies. For example, `teacher_name` is not stored in **TeacherPayroll**; it is accessed via the `teacher_id` foreign key join.

The only intentional denormalisation is the **virtual OVERDUE status** in **StudentFees**. Rather than storing OVERDUE in the database (which would require a scheduled update and create a derived-data synchronisation problem), the `FeeStatus.resolve()` enum method computes the OVERDUE status at read time by comparing `due_date` against today's date. This avoids data redundancy while providing the required business semantic.

#### 4.5. Indexing Strategy

- All foreign key columns are indexed automatically by MySQL's InnoDB engine.
- `student_code` and `teacher_code` have UNIQUE indexes, supporting fast  $O(1)$  lookup during registration.
- `token_hash` in **RememberTokens** has a UNIQUE index for fast cookie validation on every request.

- `pay_period` in `TeacherPayroll` is expected to be queried frequently; an index here would improve the monthly payroll list query (noted as a future optimisation).

## 5. Implementation Details

---

### 5.1. Authentication System

#### 5.1.1. Login Flow & Session Fixation Protection

The `LoginController` performs server-side validation before attempting authentication. On a successful credential check, it invalidates any pre-existing anonymous session to prevent session fixation attacks, then creates a fresh session with a new session ID:

```
1      // Destroy pre-login anonymous session to prevent session
      //      fixation
2      HttpSession oldSession = request.getSession(false);
3      if (oldSession != null) {
4          oldSession.invalidate();
5      }
6      // Create a brand-new session with a fresh, unpredictable ID
7      HttpSession session = request.getSession(true);
8      String csrfToken = UUID.randomUUID().toString();
9      session.setAttribute("user", user);
10     session.setAttribute("role", user.getRole().getRoleName());
11     session.setAttribute("csrfToken", csrfToken);
12     session.setMaxInactiveInterval(30 * 60); // 30-minute inactivity
      //      timeout
```

Listing 1: Session fixation protection in `LoginController.java`

A UUID CSRF token is generated and stored in the session. It is embedded in the logout form as a hidden field and validated server-side in `LogoutController.doPost`. A mismatch returns HTTP 403.

#### 5.1.2. BCrypt Password Hashing

Passwords are never stored in plain text. Registration uses `jBCrypt` to hash the password before persisting:

```
1 String rawPassword = request.getParameter("password");
2 String hashedPassword = BCrypt.hashpw(rawPassword, BCrypt.
    gensalt());
3 User newUser = new User(hashedPassword, roleId, username, email)
    ;
4 userDao.insertUser(newUser);
```

Listing 2: BCrypt hashing during registration (RegisterController.java)

Authentication in `UserDAO.authenticate()` retrieves the stored hash and uses `BCrypt.checkpw()` to verify. The CPU-intensive cost factor in BCrypt makes brute-force attacks computationally expensive.

### 5.1.3. Remember Me Implementation

The Remember Me feature stores a UUID token in both an `HttpOnly` cookie and the `RememberTokens` database table. On subsequent requests, `AuthFilter` detects a missing session but a present cookie, validates the token against the database, and auto-restores the session:

```
1 String tokenHash = UUID.randomUUID().toString();
2 // 1. Persist token in database with 30-day expiry
3 userDao.storeTokenToDatabase(user.getUserId(), thirtyDaysInSec,
    tokenHash);
4 // 2. Set HttpOnly cookie (inaccessible to JavaScript)
5 Cookie cookie = new Cookie("remember_token", tokenHash);
6 cookie.setMaxAge((int) thirtyDaysInSec);
7 cookie.setHttpOnly(true);
8 cookie.setPath("/");
9 response.addCookie(cookie);
```

Listing 3: Remember Me token storage (LoginController.java)

On logout, the token is deleted from the database and the cookie is expired (`MaxAge = 0`), ensuring no persistent access remains.

## 5.2. Role-Based Access Control (RBAC)

RBAC is enforced entirely in the filter layer, not in individual controllers. Four `@WebFilter`-annotated classes intercept requests:

```
1  @WebFilter("/payroll")
2  public class AdminFilter implements Filter {
3      @Override
4      public void doFilter(ServletRequest req, ServletResponse res
5          , FilterChain chain)
6          throws IOException, ServletException {
7          HttpSession session = ((HttpServletRequest) req).
8              getSession(false);
9          User user = (session != null) ? (User) session.
10              getAttribute("user") : null;
11          if (user == null || !user.isAdmin()) {
12              // Redirect to role-appropriate landing page
13              String redirect = (user != null && user.isTeacher())
14                  ? "/financial-statistics" : "/payment";
15              ((HttpServletResponse) res).sendRedirect(
16                  ((HttpServletRequest) req).getContextPath() +
17                      redirect);
18              return;
19          }
20          chain.doFilter(req, res);
21      }
22  }
```

Listing 4: AdminFilter role enforcement

This design means that even if a student somehow obtains the URL `/payroll`, the filter redirects them before the servlet is ever invoked.

### 5.3. Payroll Business Logic

#### 5.3.1. Automatic Net Amount Computation

The `net_amount` column in `TeacherPayroll` is a MySQL **generated column**. Its value is always `base_salary + allowances - deductions` and is recomputed by the database engine on every INSERT or UPDATE:

Listing 5: Generated column in TeacherPayroll schema

```
1  net_amount DECIMAL(12,2) GENERATED ALWAYS AS
2  (base_salary + allowances - deductions) STORED,
```

This guarantees that `net_amount` is always consistent with the components regardless of how the row is updated, eliminating the risk of stale computed values.

### 5.3.2. Duplicate Payroll Prevention

Before inserting payroll records for the current month, the controller checks whether any record for that period already exists:

```
1     if (teacherPayrollDAO.periodAlreadyGenerated(period)) {
2         request.getSession().setAttribute("flashError",
3             "Payroll for " + period + " has already been generated.");
4         response.sendRedirect("payroll?period=" + period);
5         return;
6     }
```

Listing 6: Duplicate payroll guard (PayrollController.java)

### 5.3.3. Automated Status Transition via MySQL Event

A MySQL scheduled event runs every hour and automatically transitions PENDING payroll records whose `payment_date` is in the past to ON\_HOLD:

Listing 7: MySQL scheduled event for automatic payroll escalation

```
1     CREATE EVENT IF NOT EXISTS update_passed_paydate_event
2     ON SCHEDULE EVERY 1 HOUR
3     DO
4     UPDATE TeacherPayroll
5     SET status = 'ON_HOLD'
6     WHERE status = 'PENDING'
7     AND payment_date < CURDATE();
```

This is an advanced automation feature that removes the need for an administrator to manually identify and flag overdue payrolls.

## 5.4. Student Fee Payment with Security Guard

The `payFee()` method in `PaymentController` includes a critical security check: the SQL UPDATE includes both `fee_id` AND `student_id` as conditions, preventing a student from paying another student's fee by manipulating the `feeId` parameter:

```
1 private static final String PAY_FEE =
2 "UPDATE StudentFees SET status = 'PAID', payment_date = CURDATE
3   () " +
4 "WHERE fee_id = ? AND student_id = ? AND status = 'UNPAID'";
5
6 public boolean payFee(int feeId, int studentId) throws
7   SQLException {
8   try (Connection conn = getConnection();
9     PreparedStatement stmt = conn.prepareStatement(PAY_FEE)) {
10     stmt.setInt(1, feeId);
11     stmt.setInt(2, studentId); // ownership check
12     return stmt.executeUpdate() > 0;
13   }
14 }
```

Listing 8: Student ID guard in fee payment (StudentFeeDAO.java)

The combination of `PreparedStatement` (SQL injection prevention) and the ownership guard on `student_id` ensures both correctness and security.

### 5.5. OVERDUE Status: Virtual Computation

The `FeeStatus.OVERDUE` status is never stored in the database. Instead, it is computed at read time by `FeeStatus.resolve()`:

```
1 public static FeeStatus resolve(FeeStatus stored, java.sql.Date
2   dueDate) {
3   if (stored == UNPAID && dueDate != null
4     && dueDate.toLocalDate().isBefore(LocalDate.now())) {
5     return OVERDUE;
6   }
7   return stored;
8 }
```

Listing 9: Virtual OVERDUE resolution (FeeStatus.java)

This approach avoids the complexity and potential inconsistency of a scheduled database job that would need to update thousands of rows. The fee status is always accurate as of the moment of reading.



## 5.6. Post/Redirect/Get Pattern

All POST handlers that modify data redirect after completion. Flash messages stored in the session carry user feedback across the redirect boundary:

```
1      // In POST handler -- set flash message and redirect
2      if (success) {
3          request.getSession().setAttribute("flashSuccess",
4              "Payment submitted successfully.");
5      } else {
6          request.getSession().setAttribute("flashError",
7              "Payment failed. The fee may already be paid or not found.")
8              ;
9      }
10     response.sendRedirect("payment");
11
12     // In GET handler -- read and clear flash message
13     String flashSuccess = (String) session.getAttribute("
14         flashSuccess");
15     if (flashSuccess != null) {
16         request.setAttribute("flashSuccess", flashSuccess);
17         session.removeAttribute("flashSuccess");
18     }
```

Listing 10: Flash message pattern (PaymentController.java)

This prevents duplicate form submissions when the user refreshes the browser after a payment.

## 5.7. Server-Side Form Validation

All forms perform server-side validation. The payroll edit form demonstrates the pattern – each field is validated independently, error attributes are set on the request, and the form is re-rendered (not redirected) when validation fails, preserving the user's input:

```
1      private boolean validateForm(HttpServletRequest request) {
2          boolean isValid = true;
3          String baseSalary = request.getParameter("baseSalary");
4
5          if (baseSalary == null || baseSalary.trim().isEmpty()) {
6              request.setAttribute("errorBaseSalary", "Base salary is
```

```
        required.");
7         isValid = false;
8     } else {
9         try {
10            double bs = Double.parseDouble(baseSalary);
11            if (bs < 0) {
12                request.setAttribute("errorBaseSalary",
13                    "Base salary cannot be negative.");
14                isValid = false;
15            }
16        } catch (NumberFormatException e) {
17            request.setAttribute("errorBaseSalary",
18                "Base salary must be a valid numeric value.");
19            isValid = false;
20        }
21    }
22    // ... similar blocks for allowances, deductions, status
23    return isValid;
24 }
```

Listing 11: Payroll form validation (PayrollController.java)

## 5.8. AI Assistance Report

### 5.8.1. Tools Used

- **Claude (Anthropic)** – architecture review, code review, SQL query optimisation, report drafting
- **Gemini (OpenAI)** – boilerplate code generation for DAO patterns, JSP page scaffolding
- **GitHub Copilot** – inline autocomplete during active coding sessions

### 5.8.2. Authentication System

- **AI Tool:** Gemini
- **What We Asked:** “Generate a Java Servlet login controller with session management”
- **What AI Generated:** A basic servlet that stored the username in a session attribute.

It used plain-text password comparison (`password.equals(storedPassword)`).

- **What We Modified:** Added BCrypt hash verification, session fixation protection (`oldSession.invalidate()`), CSRF token generation, role-based redirect logic, and generic error messages.
- **What We Learned:** Why session fixation is a real attack vector and how invalidating the pre-login session defeats it.

### 5.8.3. Remember Me Feature

- **AI Tool:** Claude
- **What We Asked:** “How should a secure Remember Me implementation work for a servlet-based app?”
- **AI Suggestion:** Explained the database-backed token approach, the importance of `HttpOnly` cookies, and token expiry.
- **What We Implemented:** Full token lifecycle – generation, storage, cookie dispatch, `AuthFilter` validation, and cleanup on logout. We also added the UUID generation for the token itself.
- **What We Learned:** The difference between cookie-based and server-side session storage, and why storing the token (not the session) server-side is safer.

### 5.8.4. MySQL Scheduled Event

- **AI Tool:** Gemini
- **What We Asked:** “Write a MySQL event that runs hourly and updates payroll status”
- **What AI Generated:** A valid event skeleton with incorrect `ON SCHEDULE` syntax.
- **What We Modified:** Fixed the schedule syntax, added the correct `WHERE` clause filtering on `status = 'PENDING'` and `payment_date < CURDATE()`, and tested it manually.
- **What We Learned:** How MySQL `EVENT_SCHEDULER` works and how to enable it in `my.cnf`.

### 5.8.5. Challenges with AI Code

1. AI-generated DAO code used `Statement` instead of `PreparedStatement`, introducing SQL injection vulnerabilities. We rewrote all queries to use parameterized statements.
2. AI-generated JSP snippets used outdated JSTL tag library URIs (`jakarta.sun.com`) incompatible with Jakarta EE 6+. We updated all URIs to the correct `jakarta.tags.core` namespace.
3. The generated registration flow did not handle the student-code / teacher-code lookup step. We designed and implemented that lookup and linkage logic ourselves.

### 5.8.6. Overall Impact

AI tools increased development speed by approximately 25% for repetitive boilerplate (DAO CRUD methods, HTML form scaffolding). The security implementation – session fixation, CSRF, BCrypt, prepared statements – required significant independent work to understand and apply correctly. All team members reviewed AI-generated code before committing it.

## 6. Security Implementation

---

### 6.1. Password Hashing

Passwords are hashed using **jBCrypt 0.4** with a randomly generated salt. BCrypt is a deliberately slow algorithm that increases the cost of brute-force attacks. The default work factor of 10 (applied by `BCrypt.gensalt()`) means each password check takes approximately 100–300 ms on modern hardware, making dictionary attacks impractical.

### 6.2. SQL Injection Prevention

All database interactions use `java.sql.PreparedStatement` with parameterized placeholders (?). String concatenation is never used to build SQL queries. This ensures user input is always treated as data, never as SQL code.

### 6.3. XSS Prevention

JSP pages use JSTL EL expressions (`${...}`) which perform HTML entity escaping by default, preventing stored or reflected XSS. All user-supplied data rendered in views is passed through the EL evaluator rather than inserted via `out.print()`.

### 6.4. CSRF Protection

A per-session CSRF token is generated on login. The logout form embeds this token as a hidden input. `LogoutController.doPost()` validates the submitted token against the session token before proceeding. A mismatch triggers a 403 response.

### 6.5. Authentication Security

Session fixation is addressed by invalidating the pre-login session before creating a new authenticated session. Login failure messages are generic (“Invalid username or password”) to prevent username enumeration. Sessions expire after 30 minutes of inactivity (`setMaxInactiveInterval(1800)`).

### 6.6. Authorisation

Every protected route is guarded by a dedicated filter that checks the session role before allowing the request to reach the controller. Even if a user guesses a URL, they cannot bypass the filter chain.

## 7. Testing and Quality Assurance

---

### 7.1. Testing Strategy

Testing was conducted manually using browser-based exploratory testing and SQL direct queries to verify database state. The following scenarios were executed:

| # | Test Scenario                            | Expected Result                                |
|---|------------------------------------------|------------------------------------------------|
| 1 | Login with correct credentials (admin)   | Redirect to <code>/payroll</code>              |
| 2 | Login with correct credentials (teacher) | Redirect to <code>/financial-statistics</code> |

| #  | Test Scenario                                                   | Expected Result                                  |
|----|-----------------------------------------------------------------|--------------------------------------------------|
| 3  | Login with correct credentials (student)                        | Redirect to <code>/payment</code>                |
| 4  | Login with wrong password                                       | Error message shown, no redirect                 |
| 5  | Access <code>/payroll</code> while logged in as student         | Redirect to <code>/payment</code>                |
| 6  | Access <code>/payment</code> while not logged in                | Redirect to <code>/login</code>                  |
| 7  | Register with a valid student code                              | Account created, redirect to login               |
| 8  | Register with a student code already linked to an account       | Error message “Account already exists”           |
| 9  | Register with a non-existent student code                       | Error message shown                              |
| 10 | Register with mismatched passwords                              | Error message shown, form re-rendered            |
| 11 | Generate payroll for the current period                         | Payroll records created for all teachers         |
| 12 | Generate payroll again for the same period                      | Flash error “already generated”, no duplicates   |
| 13 | Edit payroll with negative base salary                          | Validation error shown inline                    |
| 14 | Edit payroll with non-numeric allowances                        | Validation error shown inline                    |
| 15 | Pay a fee as student                                            | Fee status changes to PAID, total debt decreases |
| 16 | Attempt to pay a fee belonging to another student (direct POST) | Operation fails (student_id guard)               |
| 17 | Logout with valid CSRF token                                    | Session destroyed, redirect to login             |
| 18 | Logout with tampered CSRF token                                 | HTTP 403 returned                                |
| 19 | Check Remember Me on login, close browser, reopen               | Session restored automatically                   |

| #  | Test Scenario            | Expected Result                       |
|----|--------------------------|---------------------------------------|
| 20 | Logout after Remember Me | Cookie cleared, token deleted from DB |

All 20 test scenarios passed in the final build.

## 7.2. Bug Tracking

Bugs were tracked via GitHub Issues on the repository. Key bugs encountered and resolved during development:

- **Bug #1: JSTL EL expressions not evaluated** – JSP pages were rendering literal `${variable}` text. Root cause: missing `isELIgnored="false"` in the `@page` directive and incorrect JSTL URI. Fixed by updating the URI to `jakarta.tags.core` and adding the directive.
- **Bug #2: Student fee payment allowed for wrong student** – Early implementation only checked `fee_id`. Fixed by adding `AND student_id = ?` to the UPDATE query.
- **Bug #3: Duplicate payroll records on page refresh** – A browser refresh on the generate URL re-triggered payroll generation. Fixed by implementing the PRG pattern: redirect after generation, not forward, and adding the `periodAlreadyGenerated()` guard.
- **Bug #4: Remember Me token not cleared on logout** – Logging out did not invalidate the persistent token, allowing re-login via cookie. Fixed by adding `userDAO.deleteTokenFromDatabase(tokenHash)` in `LogoutController`.
- **Bug #5: Flash messages persisting across multiple page loads** – Flash messages were not removed after being displayed. Fixed by calling `session.removeAttribute()` immediately after reading the attribute in the GET handler.

## 7.3. Known Limitations

- **No connection pooling:** Each DAO method opens and closes its own JDBC connection via `DriverManager.getConnection()`. Under load, this would be a performance bottleneck. The recommended fix is to configure a Tomcat JNDI

DataSource (noted in `docs/SETUP.md`).

- **Hardcoded database credentials:** The `DB_URL`, `DB_USER`, and `DB_PASSWORD` constants in DAO classes should be externalised to environment variables or a `context.xml` DataSource.
- **Limited search and filtering:** The current payroll list supports period-based filtering only. Multi-criteria search across teacher name or status would improve usability.
- **AuditLog not fully activated:** The `AuditLogs` table and `AuditLog` model class are defined but the logging calls in controllers are commented out. Full audit trail activation is a planned enhancement.
- **No pagination:** Large result sets (e.g. many fee records, many payroll entries) are returned in a single query. Pagination should be added for scalability.

## 8. Deployment

---

### 8.1. Deployment Process

#### 8.1.1. Prerequisites

- Java 17+
- Apache Maven 3.8+
- Apache Tomcat 10+
- MySQL 8+
- Node.js 18+ (for Tailwind CSS compilation)

#### 8.1.2. Step-by-Step Setup

##### 1. Clone the repository:

```
git clone https://github.com/BakBackB/WebAppSchoolManagement.git
cd WebAppSchoolManagement
```



**2. Set up the database:**

```
mysql -u root -p < database/schema.sql
mysql -u root -p school_management < database/seed-data.sql
```

Enable the MySQL event scheduler in `my.cnf`: `event_scheduler=ON`.

**3. Configure the database connection:** Edit `backend/src/main/webapp/META-INF/context.xml`:

```
<Resource name="jdbc/schoolDB"
url="jdbc:mysql://localhost:3306/school_management"
username="YOUR_DB_USER"
password="YOUR_DB_PASSWORD" />
```

Also update `DB_URL`, `DB_USER`, and `DB_PASSWORD` constants in each DAO class.

**4. Build Tailwind CSS:**

```
cd frontend
npm install
npx tailwindcss -i ./src/input.css \
-o ../backend/src/main/webapp/css/output.css
```

**5. Build the WAR:**

```
cd backend
mvn clean package
```

**6. Deploy to Tomcat:** Copy `target/school_management.war` to Tomcat's `webapps/` directory and start Tomcat.**7. Access the application:**

```
http://localhost:8080/school_management/login
```

## 8.2. CI/CD Pipeline

A CI/CD pipeline was not implemented in this project. The repository is structured to support a GitHub Actions workflow in the future – the Maven build (`mvn clean package`) is self-contained and would be straightforward to automate.

## 8.3. Monitoring and Maintenance

- **Logging:** Tomcat’s built-in access logs record all HTTP requests. Application-level logging via the commented-out `AuditLogger` in controllers is prepared for activation.
- **MySQL Event Scheduler:** The `update_passed_paydate_event` runs hourly and can be monitored via `SHOW EVENTS`.
- **Session cleanup:** Tomcat automatically expires sessions after the configured 30-minute inactivity period.
- **Token cleanup:** Expired `RememberTokens` records should be periodically purged. A `DELETE FROM RememberTokens WHERE expires_at < NOW()` query scheduled as a MySQL event is the recommended approach.

# 9. User Guide

---

## 9.1. Getting Started

1. Navigate to the application URL.
2. You will be redirected to the login page at `/login`.
3. Enter your username and password. Optionally check “Remember Me” to stay logged in for 30 days.
4. After successful login, you will be automatically redirected to the landing page for your role.

To create a new account, click **Sign Up** on the login page. You will need your student or teacher code, which is assigned by the school administration.

## 9.2. Student: Fee Payment Portal

1. After logging in, you are taken to your **Fee Payment Portal**.
2. The portal displays all your fee records with their amounts, due dates, and current status (PAID / UNPAID / OVERDUE).
3. Your total outstanding balance is shown at the top of the page.
4. To pay a fee, click the **Pay** button next to an UNPAID or OVERDUE fee.
5. A success or error message will appear at the top of the page confirming the outcome.

## 9.3. Admin: Payroll Management

1. After logging in, you are taken to the **Payroll Dashboard**.
2. The header cards show the total payroll amount, count of disbursed records, pending records, and on-hold records for the selected period.
3. Use the month picker to navigate between pay periods.
4. Click **Generate Payroll for [period]** to create payroll records for all teachers in the current month. This button is only shown for the current period and only if payroll has not yet been generated.
5. Click the edit icon on any payroll card to adjust base salary, allowances, deductions, or status.
6. Click the delete icon to permanently remove a payroll record (a confirmation screen is shown first).

## 9.4. Teacher / Admin: Financial Statistics

After logging in (or navigating to `/financial-statistics`), the financial statistics dashboard displays a summary of school financial data including fee collection totals and payroll overview. This page is read-only for teachers; admins also have navigation links to the payroll management and fee payment areas.

## 10. Team Collaboration

### 10.1. Team Organisation

The team used GitHub for version control and Zalo for daily communication. Work was divided by feature domain to minimise merge conflicts:

| Member                           | Primary Responsibilities                                                       |
|----------------------------------|--------------------------------------------------------------------------------|
| Bùi Thị Thanh Ngân (ITCSIU24061) | Authentication system, filter chain, user registration, database schema        |
| Phạm Quang Thịnh (ITDSIU24047)   | Teacher payroll management, financial statistics dashboard, MySQL event        |
| Trần Minh Phúc (ITCSIU24070)     | Student fee portal, payment controller, front-end UI, Tailwind CSS integration |

All team members contributed to code review, testing, and documentation.

### 10.2. Individual Contributions

Git statistics from the repository confirm significant commit contributions from all three members.

### 10.3. Challenges in Teamwork

- **Merge conflicts on DAO classes:** Early in the project, multiple members editing the same DAO class caused conflicts. Resolved by splitting DAO files by entity and establishing a convention of one DAO per database table.
- **Database schema evolution:** Adding the `user_id` foreign key to the `Students` and `Teachers` tables mid-project required all members to re-run the schema. Resolved by maintaining a versioned `schema.sql` in the repository that could be re-applied cleanly.
- **JSTL compatibility:** Disagreement over the correct JSTL namespace for Jakarta EE 6 caused JSP rendering issues. Resolved after consulting the Jakarta EE documentation and updating all JSP files.

## 11. Conclusion and Reflection

---

### 11.1. Project Summary

The School Management System successfully delivers a secure, role-aware web application for managing student fees and teacher payroll. The core features – role-based access control, BCrypt-secured authentication, student fee portal, teacher payroll management with automated status transitions, and a financial statistics dashboard – are all implemented and tested. The application demonstrates a solid command of server-side MVC architecture, relational database design, and web security fundamentals.

Compared to the initial plan, the audit log feature remains partially implemented (the infrastructure is present but not activated in controllers), and there is no pagination on list views. These are identified as the two highest-priority future improvements.

### 11.2. Lessons Learned

#### Technical lessons:

- Security cannot be an afterthought. Session fixation, CSRF, and SQL injection vulnerabilities were all found in the initial AI-generated code. Adding security properly required understanding the attack vectors, not just copying a fix.
- MySQL generated columns and scheduled events are powerful features for keeping computed data consistent and automating state transitions at the database level.
- The Post/Redirect/Get pattern is essential for form-based web applications. Without it, browser refreshes and back-button navigation create duplicate operations and user confusion.

#### Project management lessons:

- Defining the database schema early and agreeing on it as a team contract prevented most merge conflicts. Changes to the schema should go through a review step.
- Committing frequently with descriptive messages made it easy to identify when bugs were introduced and to roll back if needed.

### 11.3. Future Enhancements

- Activate the `AuditLog` infrastructure for full audit trail (who changed what, when).
- Add pagination and multi-criteria search/filtering to payroll and fee lists.

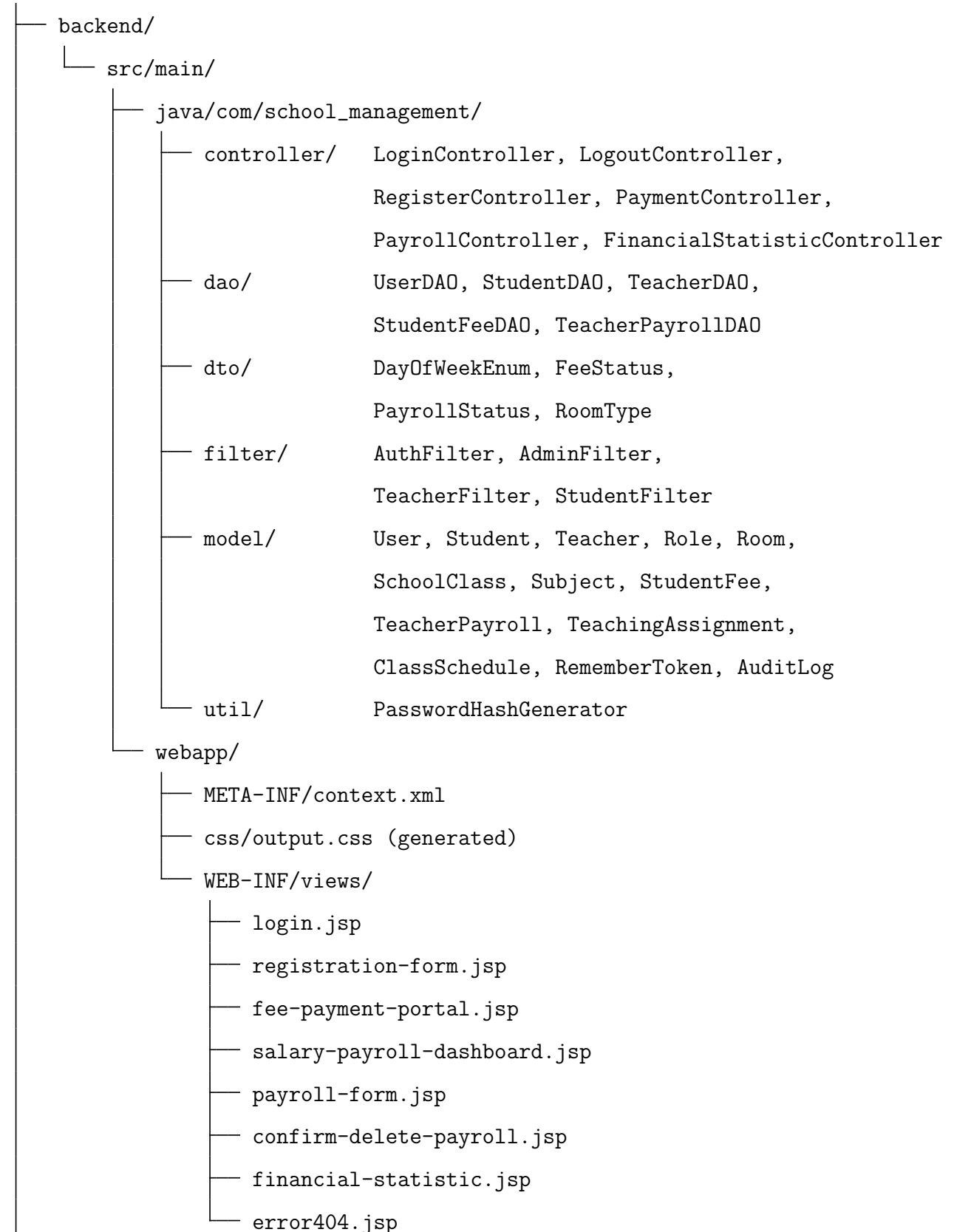
- Externalise database credentials to environment variables or a secrets manager.
- Add JDBC connection pooling via Tomcat JNDI DataSource.
- Implement email notifications (fee reminders, payroll disbursement confirmations).
- Add an admin user management panel (create/deactivate users, reassign roles).
- Containerise with Docker and Docker Compose for reproducible deployment.
- Add rate limiting on the login endpoint to prevent brute-force attacks.

## Appendix A: API / Endpoint Reference

| URL                           | Method | Access         | Description                                    |
|-------------------------------|--------|----------------|------------------------------------------------|
| /login                        | GET    | Public         | Render login form                              |
| /login                        | POST   | Public         | Process login; set session; redirect by role   |
| /logout                       | GET    | Auth           | Redirect to login with message                 |
| /logout                       | POST   | Auth           | Validate CSRF; clear session & token; redirect |
| /register                     | GET    | Public         | Render registration form                       |
| /register                     | POST   | Public         | Validate; create user; link to student/teacher |
| /payment                      | GET    | STUDENT        | List fees for current student; show total debt |
| /payment?action=pay           | POST   | STUDENT        | Mark fee as paid (with student_id guard)       |
| /payroll                      | GET    | ADMIN          | List payroll for selected period with counters |
| /payroll?action=generate      | GET    | ADMIN          | Generate payroll for current month             |
| /payroll?action=edit          | GET    | ADMIN          | Render payroll edit form                       |
| /payroll?action=confirmDelete | GET    | ADMIN          | Render delete confirmation                     |
| /payroll?action=update        | POST   | ADMIN          | Validate and save payroll changes              |
| /payroll?action=delete        | POST   | ADMIN          | Delete payroll record permanently              |
| /financial-statistics         | GET    | ADMIN, TEACHER | Render financial statistics dashboard          |

## Appendix B: Git Repository Structure

WebAppSchoolManagement/





```
— database/
  |— schema.sql
  |— seed-data.sql
  |— ER-diagram.png
  |— school.mwb
— docs/
  |— API.md
  |— ARCHITECTURE.md
  |— CONTRIBUTING.md
  |— DEPLOYMENT.md
  |— SETUP.md
— frontend/
  |— src/input.css
  |— tailwind.config.js
  |— package.json
— .gitignore
— LICENSE
— README.md
```